

Story19 - 保存实时血压数据

1. 1. Story功能描述

1.1. 1.1 系统名称

生命体征监测系统 (Vital Signs Monitor System)

1.2. 1.2 系统类型

医疗健康监测软件系统

1.3. 1.3 项目定位

本功能是生命体征监测系统中的数据持久化模块，作为系统基础支撑功能，负责各项生命体征数据的定时采集与存储。

1.4. 1.4 任务目标

- 实现体温、血氧、血压、心率数据的定时采集（每分钟一次）
- 确保数据的可靠存储和完整性
- 支持多患者数据并发存储
- 保证数据存储的实时性和准确性

2. 2. 详细设计

2.1. 2.1 界面设计

本功能为后台服务，无直接界面展示，但需要提供：

- 数据存储状态监控
- 存储异常告警提示
- 系统日志记录

2.2. 2.2 内部处理流程

1. 数据采集与存储流程：

1 | 定时触发(1分钟) -> 采集数据 -> 数据验证 -> 格式转换 -> 存储数据库

2. 异常处理流程：

1 | 异常检测 -> 日志记录 -> 重试机制 -> 告警通知

2.3. 2.3 硬件接线

- 数据库服务器连接
 - 支持高速网络连接
 - 最低带宽要求：5Mbps
 - 稳定的电源供应
- 数据采集设备连接
 - 串口/网络接口
 - 实时数据传输

2.4. 2.4 数据流



2.5. 2.5 算法描述

1. 数据采集定时器:

```
1 class DataCollectionTimer {
2     private timer: number;
3     private interval: number = 60000; // 1分钟
4
5     startCollection() {
6         this.timer = setInterval(() => {
7             this.collectAndSaveData();
8         }, this.interval);
9     }
10
11     async collectAndSaveData() {
12         try {
13             const vitalSigns = await this.collectVitalSigns();
14             await this.validateData(vitalSigns);
15             await this.saveToDatabase(vitalSigns);
16         } catch (error) {
17             this.handleError(error);
18         }
19     }
20 }
```

2. 数据验证算法:

```
1 class DataValidator {
2     validateVitalSigns(data: VitalSignsData): boolean {
3         return this.validateTemperature(data.temperature)
4             && this.validateBloodPressure(data.bloodPressure)
```

```

5      && this.validateHeartRate(data.heartRate)
6      && this.validateOxygen(data.oxygen);
7  }
8
9  private validateTemperature(temp: number): boolean {
10     return temp >= 35 && temp <= 42;
11 }
12
13 private validateBloodPressure(bp: BloodPressure): boolean {
14     return bp.systolic >= 60 && bp.systolic <= 200
15         && bp.diastolic >= 40 && bp.diastolic <= 130;
16 }
17 }

```

2.6. 2.6 外部接口

1. 数据存储接口:

```

1  interface VitalSignsData {
2      patientId: string;
3      timestamp: number;
4      temperature?: number;
5      bloodPressure?: {
6          systolic: number;
7          diastolic: number;
8      };
9      heartRate?: number;
10     oxygen?: number;
11 }
12
13 interface DataStorage {
14     saveVitalSigns(data: VitalSignsData): Promise<void>;
15     saveTemperature(patientId: string, value: number): Promise<void>;
16     saveBloodPressure(patientId: string, systolic: number, diastolic:
17 number): Promise<void>;
18     saveHeartRate(patientId: string, value: number): Promise<void>;
19     saveOxygen(patientId: string, value: number): Promise<void>;
20 }

```

2. 错误处理接口:

```

1  interface ErrorHandler {
2      handleStorageError(error: Error): void;
3      handleValidationError(error: ValidationError): void;
4      handleDeviceError(error: DeviceError): void;
5  }

```

2.7. 2.7 其他设计考虑

2.7.1. 2.7.1 性能设计

- 数据库优化
 - 分表策略（按时间、患者ID分表）
 - 索引优化
 - 批量插入
 - 连接池管理

2.7.2. 2.7.2 安全设计

- 数据安全
 - 访问权限控制
 - 数据加密存储
 - 操作日志记录
 - 数据备份策略

2.7.3. 2.7.3 可靠性设计

- 存储可靠性
 - 数据一致性检查
 - 自动重试机制
 - 数据备份机制
 - 故障转移策略

2.7.4. 2.7.4 扩展性设计

- 系统扩展
 - 支持新增数据类型
 - 可配置的存储周期
 - 可扩展的存储介质
 - 分布式存储支持